EE5203 L07 Study Sheet

Timers and non-blocking time - Basri Erdogan

Essential question: How can a program keep exact time without ever standing still — and how does a counter made of hardware become a millisecond you can use in C?

What you should be able to do

- Trace the pipeline: timer clock → `PSC` → `CNT` → `ARR` → update event.
- Compute `PSC` and `ARR` for a required period, applying the +1 rule.
- Consume an update event two ways: polling `UIF`, and by interrupt.
- Build a 1 ms tick and schedule work from it, wrap-safe.
- Prove in the register view that the counter runs independently of your loop.

The pipeline, with real names

```
timer clock (16 MHz)
     ->  PSC          divide by (PSC + 1)      -> counting frequency
     ->  CNT          counts up, one step per counting tick
     ->  ARR          when CNT reaches ARR, CNT wraps to 0
     ->  update event UIF set in TIM2->SR
     ->  UIE in DIER  event allowed to raise an interrupt
     ->  TIM2_IRQn    NVIC delivers it
     ->  TIM2_IRQHandler()               (processor enters this)
     ->  HAL_TIM_IRQHandler()            (clears UIF, then...)
     ->  HAL_TIM_PeriodElapsedCallback() (your code - HAL calls this)
```

This is L06's interrupt path with one box changed. The button raised the request there; the counter raises it here. Everything after `NVIC` is identical.

The two divisions

$$f_{\text{update}} = \frac{f_{\text{timer clock}}}{(\text{PSC} + 1) \times (\text{ARR} + 1)}$$

Both divisions use the register value plus one. `PSC = 0` divides by 1; `ARR = 999` means the counter visits 1000 states, `0` through `999`. A missing `+1` makes every period slightly wrong — and the error is small enough to survive a casual test, which is what makes it a classic bug.

Requirement (16 MHz clock)	`PSC`	`ARR`	Working
1 ms tick	15	999	16 MHz ÷ 16 = 1 MHz; 1 MHz ÷ 1000 = 1 kHz
200 ms, 1 kHz counting	15999	199	16 MHz ÷ 16000 = 1 kHz; ÷ 200 = 5 Hz
200 ms, 10 kHz counting	1599	1999	same period, finer counting resolution

The pair is not unique: trade counting resolution against `ARR` range.

Vocabulary table

Name	Job
`PSC`	prescaler — divides the timer clock before the counter
`CNT`	the counter itself; readable at any time
`ARR`	auto-reload — the value `CNT` wraps at
`UIF` (`SR` bit 0)	update flag; hardware sets it, software clears it
`UIE` (`DIER` bit 0)	lets the update event raise an interrupt
`CEN` (`CR1` bit 0)	counter enable — start/stop

Name	Job
`UG` (`EGR` bit 0)	force an update now (loads `PSC`/`ARR` immediately)

Consuming the event

```
/* Polling: two operations, both required */
if (TIM2->SR & TIM_SR_UIF) {    /* 1. test the flag  */
    TIM2->SR &= ~TIM_SR_UIF;    /* 2. clear it       */
    /* ... one period has elapsed ... */
}

/* Interrupt: HAL clears UIF for you before calling this */
volatile uint32_t g_ms = 0;

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM2) {   /* one callback serves every timer */
        g_ms++;                     /* record - nothing else          */
    }
}
```

Start it with `HAL_TIM_Base_Start_IT(&htim2);` — `Start` alone runs the counter but never enables `UIE`, so the callback stays silent.

- **Test *and* clear.** A polling loop that forgets the clear sees the flag still set on the next pass and fires continuously.
- **Callback rules, unchanged from L06:** check the source, record, return fast. A callback that takes 2 ms cannot keep up with a 1 ms tick.
- **`volatile`:** the interrupt changes `g_ms` between two main-loop instructions.

Scheduling from the tick

```
uint32_t t_phase = 0;
uint8_t  led_on  = 0;

while (1) {
    uint32_t wait = led_on ? 100U : 900U;
    if ((g_ms - t_phase) >= wait) {   /* unsigned: wrap-safe */
        t_phase = g_ms;
        led_on = !led_on;
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5,
                          led_on ? GPIO_PIN_SET : GPIO_PIN_RESET);
    }
}
```

The unsigned subtraction `g_ms - t_phase` stays correct across wrap-around — the same arithmetic that made L06's debounce safe.

Drift. `t_phase = g_ms` re-anchors to the moment the toggle actually happened, so every late pass pushes all later toggles: error accumulates. `t_phase += 500U` keeps the deadlines anchored to the original schedule: one toggle is late, the rest are not. Choose deliberately.

Bare metal, register by register

```
RCC->APB1ENR |= (1U << 0);   /* TIM2 clock - APB1, not AHB1        */
TIM2->PSC = 15;              /* 16 MHz / 16 = 1 MHz: 1 tick = 1 us */
TIM2->ARR = 0xFFFFFFFFU;     /* free-run: TIM2 is a 32-bit timer   */
TIM2->EGR = (1U << 0);       /* UG: load PSC now, not next update  */
TIM2->CR1 |= (1U << 0);      /* CEN: start counting                */
```

`PSC` and `ARR` are *preloaded* — the hardware copies them into their working registers at the next update event. Writing `UG` forces that copy immediately, which is why a fresh timer needs it. At 1 MHz a 32-bit counter wraps after about **71 minutes**.

SFR evidence

Claim	Evidence
prescaler and period configured	`TIM2->PSC` = 15, `TIM2->ARR` = 999
the counter is running	`TIM2->CNT` holds a different value each time you pause
update event reached the flag	`TIM2->SR` bit 0 (`UIF`) = 1 while paused before the clear
interrupt is armed	`TIM2->DIER` bit 0 (`UIE`) = 1
clock actually enabled	`RCC->APB1ENR` bit 0 = 1

Common mistakes

- Forgetting a `+1` in either division.
- `HAL_TIM_Base_Start()` instead of `..._Start_IT()` — counter runs, callback never does.
- NVIC checkbox for TIM2 left unchecked: `UIF` sets, nothing is delivered.
- Polling loop tests `UIF` but never clears it.
- Callback doing application work instead of recording.
- Missing `volatile` on `g_ms`.
- Enabling the clock on `AHB1ENR` — TIM2 lives on APB1.
- Bare-metal setup with no `UG` write, so the old prescaler survives one period.

Mastery self-check

☐ I can draw the pipeline and name every stage.
☐ I can derive `PSC`/`ARR` for a stated period and check it with the formula.
☐ I can give a second valid pair for the same period and say what changed.
☐ I can name the two operations a polling loop must do on `UIF`.
☐ I can explain who clears `UIF` in interrupt mode.
☐ I can justify `volatile` and the unsigned subtraction in one sentence each.
☐ I can predict `TIM2->CNT` and `TIM2->SR` at a breakpoint.
☐ I can explain why `EGR` is written in the bare-metal setup.

Five review questions

1. From the 16 MHz timer clock, give `PSC`/`ARR` for a 25 ms update event at a 10 kHz counting frequency. Then give a second pair for the same period and state what you traded.
2. A student configures TIM2 correctly, calls `HAL_TIM_Base_Start(&htim2)`, and reports "the counter runs but my callback never fires." Which register bit is wrong, and which one call fixes it?
3. Your polling loop toggles the LED far too fast, and `TIM2->SR` bit 0 reads 1 every time you pause. What single line is missing?
4. The tick is 1 ms. The callback is changed to write the LED and blink an external one, taking about 1.4 ms. Describe what happens to `g_ms` and to the timing of everything scheduled from it.
5. Two schedulers both toggle every 500 ms; one uses `t = g_ms`, the other `t += 500U`. One loop pass takes 30 ms. Describe the behavior of each over the following ten toggles.

See the L07 worksheet for extended practice. Worked solutions are released after the practice window.